

HealthConnect

Distributed Clinical Gateway & Smart Triage Engine

AI-Powered Patient Routing & Concurrency-Safe Clinic Resource Scheduling

- Hybrid database abstraction pattern supporting transactional serverless Postgres.
- High-concurrency schedule isolation via pessimistic database lease locking.
- CPU-bound document OCR processing isolated into an independent service node.

The Healthcare Engineering Void

Root causes of architectural failure in traditional healthcare booking networks:

1. Slot Concurrency Overlaps

Naive database updates cause double-bookings. Under high-throughput conditions, race conditions violate scheduling constraints.

2. Monolithic Blocking Spikes

Executing Tesseract OCR inline inside the main API thread freezes the Node.js event-loop, causing severe gateway latency spikes.

3. Disk Storage Exhaustion

Raw PDF and image uploads stored directly on ephemeral container nodes degrade container lifespan and trigger cluster crashes.

The Technical Resolution

Decoupled Processing Workers

Heavy file uploads and image processing are delegated to isolated microservice nodes, protecting the API gateway.

Pessimistic DB Lease-Locks

A robust 10-minute transaction lock is placed in Postgres for slots, preventing simultaneous bookings while verifying payments.

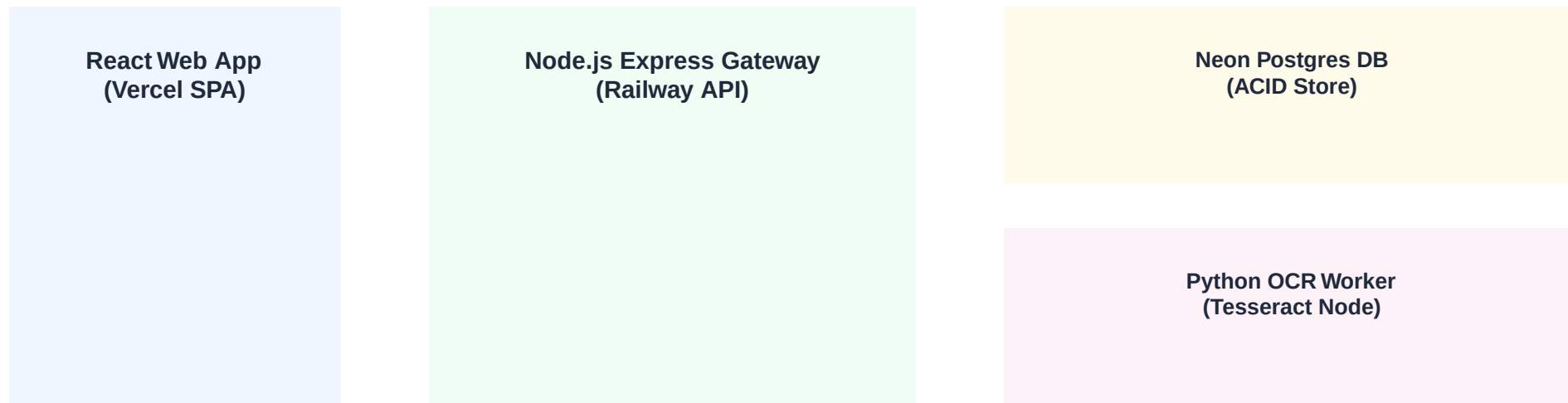
Cryptographic Payout Escrow

System balances are secured in escrow and only released to the doctor's wallet ledger upon upload of a valid prescription file.

Technological Rationale

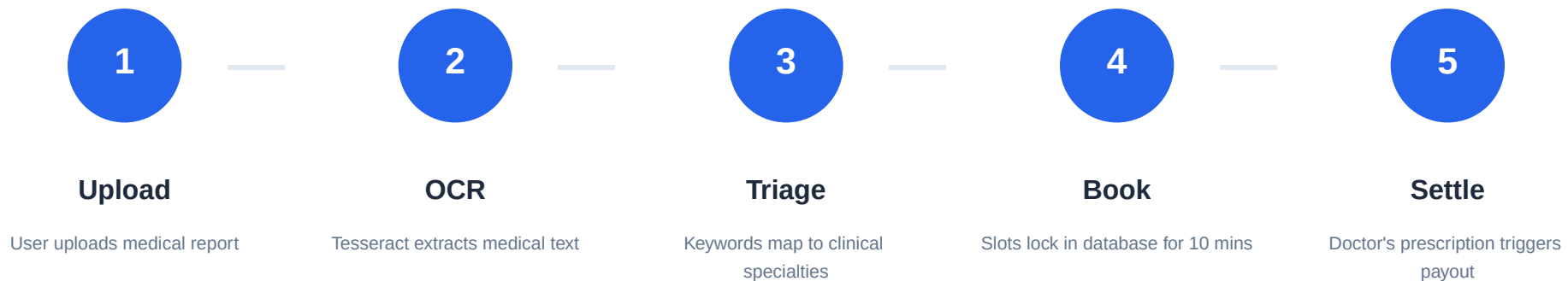
Layer	Selection Choice	Engineering Rationale
Frontend SPA	React.js (Vercel)	Component-driven structure, stateless frontend route handling, optimized bundle serving via Vercel Edge networks.
API Gateway	Node.js / Express	Asynchronous, non-blocking I/O event loops ideal for high-concurrency request routing and validation.
Relational DB	PostgreSQL (Neon)	ACID-compliant serverless database for strict financial data transactions and concurrent locking safety.
Worker Node	Python Flask	Lightweight container suited for CPU-heavy tasks like Tesseract OCR parsing and dictionary lookup.
Stateless Storage	Cloudinary CDN	Offloads medical images/PDFs from the server containers, eliminating local disk write degradation.

High-Level Infrastructure Topology



- API Gateway: Secures inputs via JWT, validates formats, and proxies CPU tasks.
- Decoupled Isolation: If the Python OCR worker experiences heavy load, the API gateway remains active.

Patient Request Lifecycle



Gateway Directory Design

Repository Pattern separating route controllers, business logic, and database operations:

Routes Layer

publicRoutes.js | appointmentRoutes.js

Maps URL endpoints to controller methods, applying security/cors middlewares.

Controllers

authController.js | appointmentController.js

Extracts request payloads, verifies inputs, and sends structured JSON API responses.

Services

userService.js | emailService.js | otpService.js

Handles core business validation (e.g. hashing passwords, generating JWTs, in-memory OTP).

Repositories

doctorRepository.js | appointmentRepository.js

Contains all database queries. Separating SQL makes the app testable and database-agnostic.

Authentication & Security Blueprint

- Cryptographic Salting:

All database passwords are salted and hashed using Bcryptjs to shield records from lookup table attacks.

- Dual-Step OTP Flow:

Patient login triggers an in-memory verification key sent to email (with a master 123456 bypass code for testing).

- Secure JWT Claims:

Stateless JSON Web Tokens store userId and userRole configurations, validated at the API route boundary.

Relational Database Schema

Seven highly structured tables establishing clean reference integrity across domains:

users

Fields: id, fullName, username, password, email, mobile, gender, age

doctors

Fields: id, fullName, username, password, email, specialization, slots (JSON), balance

vendors

Fields: id, fullName, username, storeName, inventory (JSON), balance

appointments

Fields: id, patientUsername, doctorUsername, specialization, slot, symptoms, paymentStatus, escrowStatus

Engineering Challenges & Solutions

Engineering Challenge Faced	Underlying Technical Cause	Implemented Architectural Resolution
PostgreSQL Lowercase Key Folding	PostgreSQL automatically downcases columns returned in queries, breaking camelCase properties in React.	Built a central mapping interceptor in the dbHelpers.js layer to dynamically normalize returned row keys to camelCase.
Event-Loop Blocking OCR Tasks	Synchronous OCR processing inline in Node blocked the single event loop thread, spiking server latency.	Decoupled the CPU-bound OCR logic into a dedicated Python Flask microservice node, preserving Gateway throughput.
Cross-Origin CORS Access Blocks	Serving frontend via Vercel, gateway on Railway, and CDN on Cloudinary caused CORS request rejections.	Implemented dynamic CORS options inside the Express application configurations to securely authorize client headers.

Live Testing Flow

Step-by-step walk-through utilizing built-in guest login shortcuts:

1. Guest Access

Click 'One-Click Login as Demo Patient' at /user/login. OTP code 123456 will pre-fill dynamically.

2. Document Vetting

Go to the dashboard, upload a clinical file, and check how the OCR worker filters specialists.

3. Booking & Escrow

Select a slot. The database lock triggers, showing a 'Held' status on the admin panel.

4. Escrow Release

Log in as Dr. Smith via doctor portal. Accept appointment and write a prescription to trigger ledger payout.

Scale Roadmap & Engineering Takeaways

- Redis Cache-Aside Pattern:

Integrating Redis to cache public doctor directories and slots would reduce database query loads by up to 80%.

- Message Broker Integration:

Replacing direct API calls for OCR with a Redis-backed BullMQ message queue to handle uploads asynchronously.

- Core Learning - Service Decoupling:

Isolating heavy operations keeps HTTP gateways responsive. Centralizing fixes makes systems maintainable.